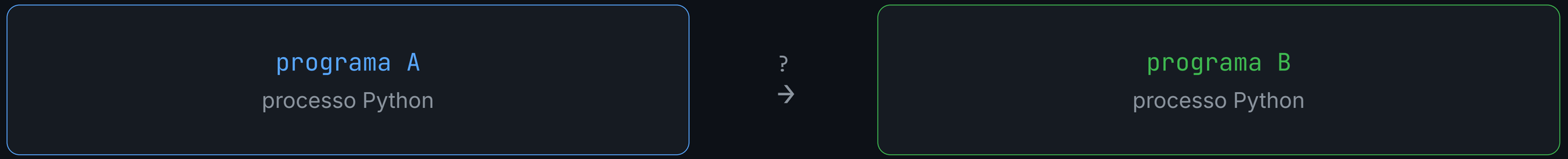
Como dois programas trocam dados?



Como um programa Python envia dados para outro programa?

Socket: a interface do programa com a comunicação

Socket é um recurso administrado pelo sistema operacional que permite a um processo enviar e receber dados.

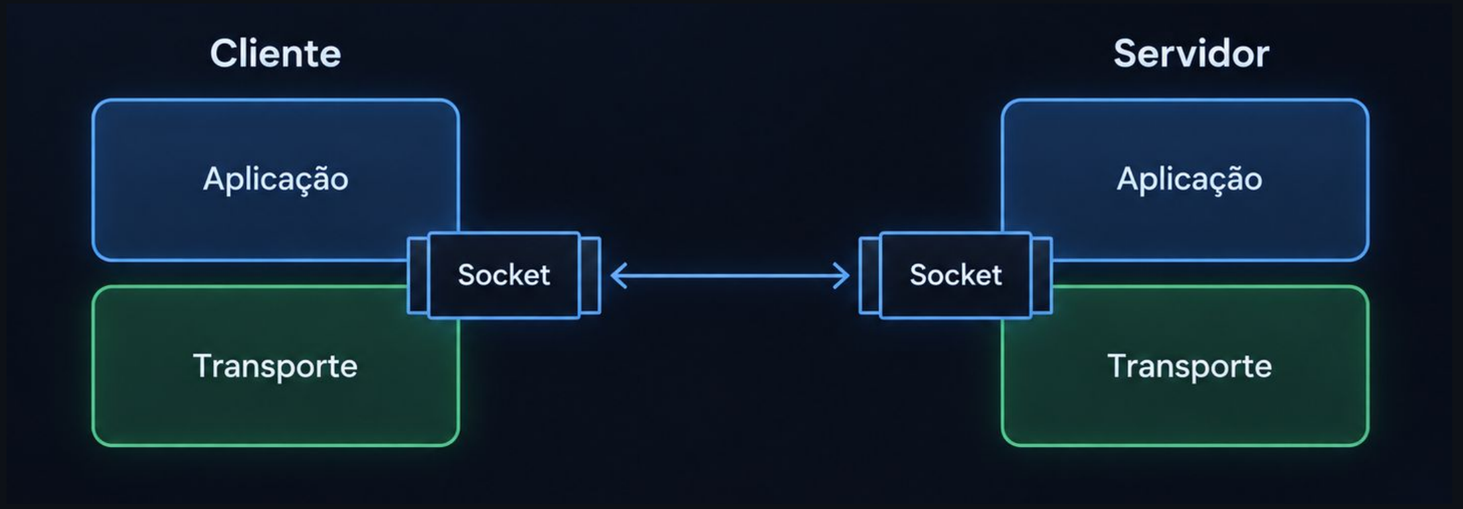
O programa usa a API de sockets do sistema operacional.

Cada ponta usa um socket.

Criar o socket não estabelece uma conexão.

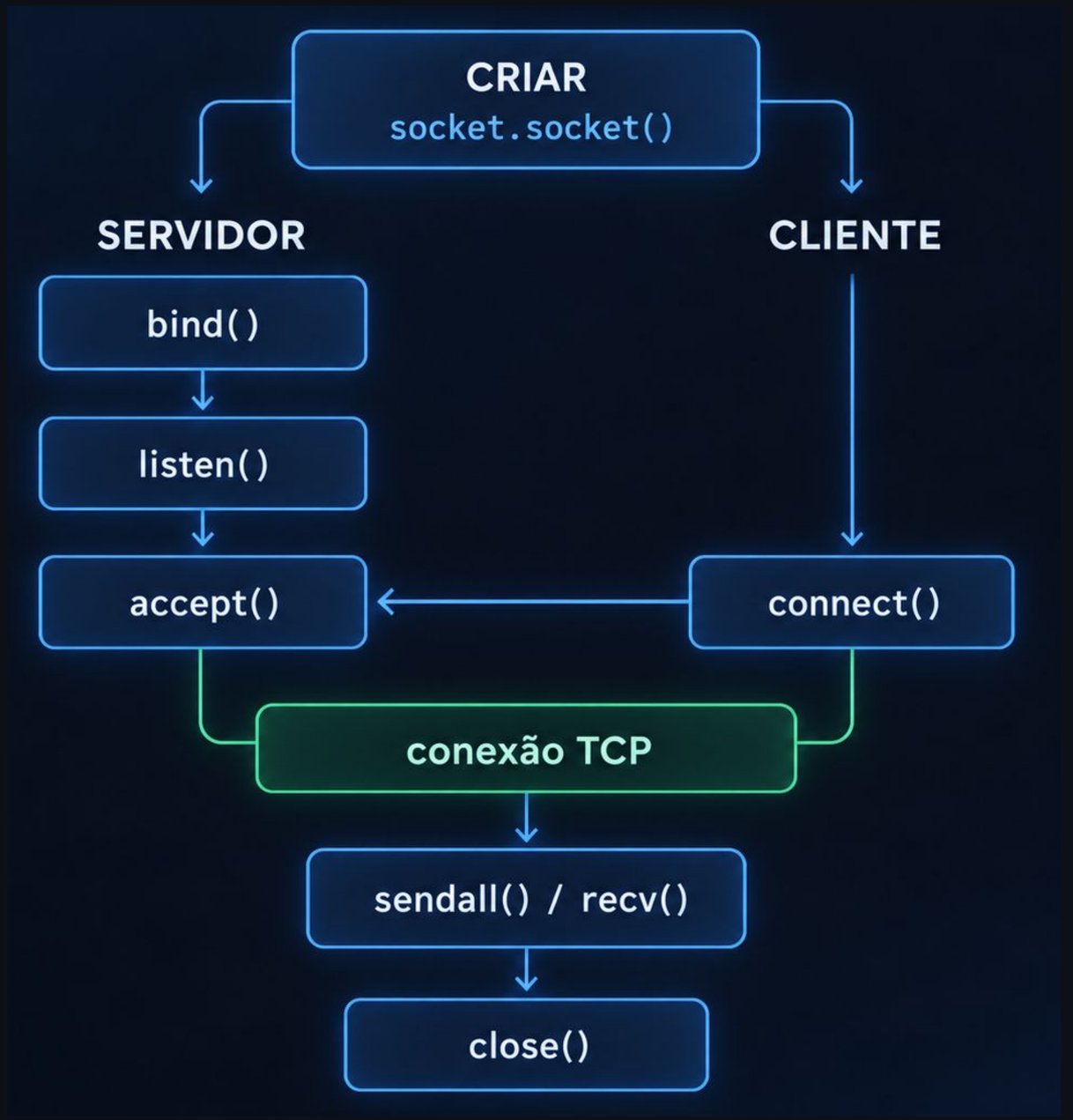
```
// não confundir
```

Socket não é protocolo, endereço IP, porta nem WebSocket.



Principais operações com sockets

Operação	O que faz
socket.socket()	Cria o socket que será usado pelo programa
bind()	Associa o socket do servidor a um IP e uma porta
listen()	Coloca o servidor em modo de espera por conexões
accept()	Aceita um cliente que se conectou
connect()	Cliente solicita conexão com o servidor
sendall()	Entrega bytes para serem enviados pela conexão
recv()	Obtém bytes recebidos pelo socket
close()	Fecha o socket



// retomada

O servidor HTTP já usava sockets por nós



`http.server` escondia quase todas as operações. Agora escreveremos cliente e servidor próprios, na porta TCP `9000`.

// checagem

Abrir `127.0.0.1:9000` no Chrome garante uma página?

// objetivo da aula

O objetivo é fazer a comunicação funcionar



// 01

Do processo à rede

O programa acessa a comunicação por meio de um socket.

O sistema operacional realiza as operações



A aplicação solicita; o sistema operacional executa as operações de rede.

O programa solicita operações através da API de sockets; o sistema operacional executa a comunicação.

Socket, IP, porta e protocolo

ELEMENTO	FUNÇÃO NO LABORATÓRIO
Socket	Recurso usado pelo processo
IP	Identifica a interface de destino
Porta	Direciona ao serviço
TCP	Serviço de transporte

Os quatro cooperam: nenhum substitui o outro.

127.0.0.1:9000

`127.0.0.1` — loopback: o próprio computador.

`9000` — porta TCP escolhida para a aula.

```
// laboratório
```

Nenhum computador de outro aluno será acessado.

Principais Tipos de Sockets

- **Sockets de Fluxo (Stream - TCP):**
  - ❏ Orientados à conexão e confiáveis;
  - ❏ Garantem que os dados cheguem em ordem correta e sem perdas;
  - ❏ Usados em navegação web (HTTP/HTTPS) e SSH.

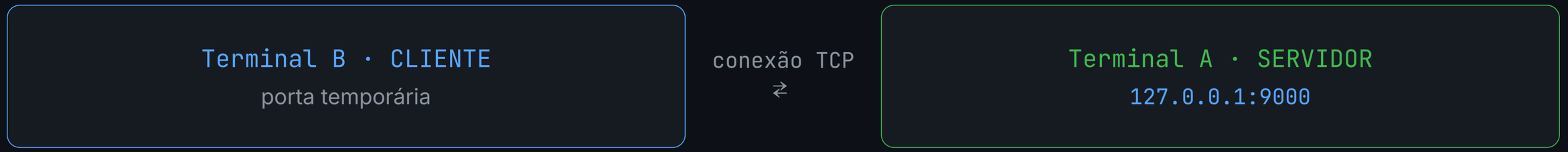
- **Sockets de Datagrama (Datagram - UDP):**
  - ❏ Não são orientados à conexão e não garantem a entrega ou a ordem dos pacotes.
  - ❏ Oferecem maior velocidade, sendo ideais para streaming de vídeo ou jogos online.

- **Unix Domain Sockets:**
  - ❏ Utilizados para a comunicação entre processos exclusivamente dentro da mesma máquina (IPC) oferecendo alta performance sem passar pela pilha de rede tradicional.

Dois terminais mostram as duas pontas

CLIENTE	SERVIDOR
Inicia a conexão	Aguarda a conexão
Envia uma mensagem	Recebe a mensagem
Espera uma resposta	Produz a resposta

Os dois processos podem executar no mesmo computador.



Durante toda a aula: terminal A = servidor, terminal B = cliente.

Um socket TCP começa com duas escolhas

03_cliente_tcp.py — linhas 1 e 10

```python
1  import socket
⋮
10 cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

`AF_INET` → IPv4.

`SOCK_STREAM` → TCP, nessa combinação.

// saída esperada

A criação não conecta, não envia e não imprime nada.

O servidor prepara o atendimento

`socket()` cria o recurso → `bind()` associa a 127.0.0.1:9000 → `listen()` prepara a espera → `accept()` aceita o cliente

O cliente solicita a conexão

`socket()` cria o recurso → `connect()` pede a conexão → `enviar` sendall() → `receber` recv() → `close()` encerra

// checagem

O que acontece se o servidor não estiver executando?

A conexão acontece antes dos dados

01 Servidor executa e espera.

02 Cliente executa e conecta.

03 Servidor aceita.

04 Cliente envia.

05 Servidor responde.

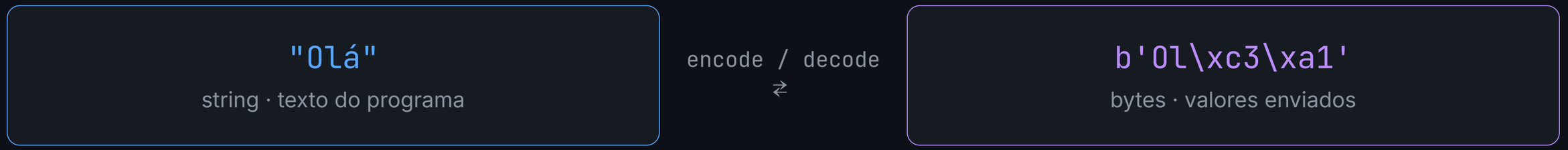
06 Cliente recebe.

> // checagem
> Qual programa deve ser iniciado primeiro?

String e bytes são tipos diferentes

O socket envia bytes

Antes de enviar um texto, o programa precisa transformá-lo em bytes.



> `\xc3\xa1` são dois bytes em hexadecimal: juntos formam o “á”.

O primeiro script converte e recupera o texto

01_texto_bytes.py — linhas 1–13

```python
texto = "Olá"

# String → bytes
dados = texto.encode("utf-8")

print("Texto:", texto)
print("Bytes:", dados)
print("Lista de bytes:", list(dados))

# Bytes → string
texto_recuperado = dados.decode("utf-8")

print("Texto recuperado:", texto_recuperado)
```

“Olá” = 3 caracteres.

Em UTF-8 = 4 bytes.

`list(dados)` mostra cada byte como número.

A execução mostra os bytes de “Olá”

Terminal B — saída esperada

```
PS> py 01_texto_bytes.py
Texto: Olá
Bytes: b'Ol\xc3\xa1'
Lista de bytes: [79, 108, 195, 161]
Texto recuperado: Olá
```

79 = O · 108 = l · 195 e 161 = á

// checagem

O socket enviará a string ou os bytes?

O helper envia os bytes

protocolo_tcp.py — enviar_mensagem() · linhas 1-3

```python
def enviar_mensagem(socket_conexao, mensagem):
    dados = mensagem.encode("utf-8")
    socket_conexao.sendall(dados)
```

`socket_conexao` : o socket da conversa.

`encode`: texto → bytes.

`sendall` : tenta enviar todos os bytes ou informa falha.

// atenção

Concluir `sendall()` não confirma processamento remoto.

O helper recebe bytes e recupera o texto

protocolo_tcp.py — receber_mensagem() · linhas 6-12

```python
def receber_mensagem(socket_conexao):
    dados = socket_conexao.recv(1024)

    if not dados:
        return None

    return dados.decode("utf-8")
```

`recv(1024)` solicita até 1024 bytes.

Bytes vazios → fim do fluxo TCP → `None`.

`decode` recupera o texto observado.

```
// por enquanto
```

Usaremos mensagens curtas e uma troca por vez.

O servidor usa a porta 9000

02_servidor_tcp.py — linhas 1–15

```
1  import socket
2
3  from protocolo_tcp import enviar_mensagem, receber_mensagem
4
5
6  HOST = "127.0.0.1"
7  PORTA = 9000
8
9
10 servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
11
12 servidor.bind((HOST, PORTA))
13 servidor.listen()
14
15 print(f"Servidor esperando em {HOST}:{PORTA}...")
```

Linha 10: cria o socket TCP.

Linha 12: associa a `127.0.0.1:9000`.

Linha 13: prepara a espera.

O servidor aceita, recebe e responde

02_servidor_tcp.py — linhas 18-39

```
18  conexao, endereco = servidor.accept()
    ⋮
23  while True:
24
25      mensagem = receber_mensagem(conexao)
26
27      if mensagem is None:
28          print("Cliente desconectou.")
29          break
    ⋮
33      resposta = "ECO: " + mensagem
34
35      enviar_mensagem(conexao, resposta)
    ⋮
38  conexao.close()
39  servidor.close()
```

`conexao` representa o cliente aceito.

`None` → cliente desconectou → sai do laço.

Os dois `close()` liberam os sockets.

O cliente conecta ao servidor

03_cliente_tcp.py — linhas 6-14

```
6  HOST = "127.0.0.1"
7  PORTA = 9000
8
9
10 cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
11
12 cliente.connect((HOST, PORTA))
13
14 print("Conectado ao servidor!")
```

Endereço e porta precisam coincidir com os do servidor.

O cliente não usa `bind` nem `listen`.

O cliente envia e espera o eco

03_cliente_tcp.py — linhas 17-31

```python
while True:

    mensagem = input("Mensagem: ")

    if mensagem == "sair":
        break

    enviar_mensagem(cliente, mensagem)

    resposta = receber_mensagem(cliente)

    print("Servidor:", resposta)


cliente.close()
```

01 ler

02 verificar `sair`

03 enviar

04 aguardar

05 imprimir

A primeira comunicação usa dois terminais

Terminal A · servidor — exemplo esperado

```
PS> py 02_servidor_tcp.py
Servidor esperando em 127.0.0.1:9000...
Cliente conectado: ('127.0.0.1', 53117)
Recebido: Olá
Recebido: Redes II
Recebido: 123
Cliente desconectou.
```

Terminal B · cliente — exemplo esperado

```
PS> py 03_cliente_tcp.py
Conectado ao servidor!
Mensagem: Olá
Servidor: ECO: Olá
Mensagem: Redes II
Servidor: ECO: Redes II
Mensagem: 123
Servidor: ECO: 123
Mensagem: sair
```

> Para testar de novo: reiniciar primeiro o servidor, depois o cliente.

// atividade

A primeira troca TCP

01 Abrir dois terminais na pasta dos arquivos.

02 Terminal A: `py 02_servidor_tcp.py`.

03 Terminal B: `py 03_cliente_tcp.py`.

04 Enviar seu nome, uma palavra com acento e uma frase curta.

05 Conferir `ECO:`, digitar `sair` e observar o encerramento.

registro

```
Mensagem enviada:
Resposta recebida:
Qual processo começou primeiro?
O que encode e decode fizeram?
```

// ok

Comunicação executada e papéis explicados.

// resumo

A primeira comunicação funcionou

01 O servidor aguardou em `127.0.0.1:9000`.

02 O cliente estabeleceu a conexão.

03 O texto virou bytes.

04 O servidor recebeu e respondeu.

05 O cliente mostrou `ECO:`.

06 `sair` encerrou o cliente; o servidor observou a desconexão.

// conclusão

Dois processos Python trocaram dados por sockets TCP.

O que é TCP e o que ele faz?

TCP atua abaixo do socket: cria uma conexão lógica e transporta um fluxo confiável de bytes entre cliente e servidor.

Aplicação Python
decide o que enviar

↓

Socket
endpoint usado pelo processo

↓

TCP
transporte confiável de bytes

↓

IP
endereçamento e rota

↓

Rede
meio físico/lógico

1. Estabelece a conexão
Antes dos dados, cliente e servidor estabelecem uma comunicação TCP.

2. Entrega em ordem
O receptor obtém o fluxo de bytes na sequência correta.

3. Detecta perdas
Se dados se perdem no caminho, o TCP pode retransmitir.

4. Não entende mensagens
Para o TCP, tudo é um fluxo contínuo de bytes.

Exemplo na aula
"Olá" → encode → bytes → sendall() → TCP → recv() → decode

Ideia-chave: TCP transporta bytes; a aplicação define onde uma mensagem começa e termina.

Resumo: o socket é a interface usada pela aplicação; o TCP é o protocolo que transporta os bytes entre os endpoints.

// para pensar

Será que `recv()` sempre devolve uma mensagem?

protocolo_tcp.py — linha 7

```
7       dados = socket_conexao.recv(1024)
```

01 Se o cliente enviar dois textos rapidamente, quantas leituras ocorrerão?

02 Uma mensagem grande pode exigir mais de uma leitura?

03 Como o receptor saberá onde uma mensagem termina?

> Próxima aula: TCP como fluxo de bytes, quebra de linha como delimitador e uma nova versão de `protocolo_tcp.py`.